



Simran Bedi

@psimran

Closures in Javascript



What are Closures?

- Definition: Closures are a combination of functions and the lexical environment within which they were declared.
- They allow functions to retain access to variables from their outer scope, even after the outer function has finished executing.
- Closures are created whenever a function is defined within another function.



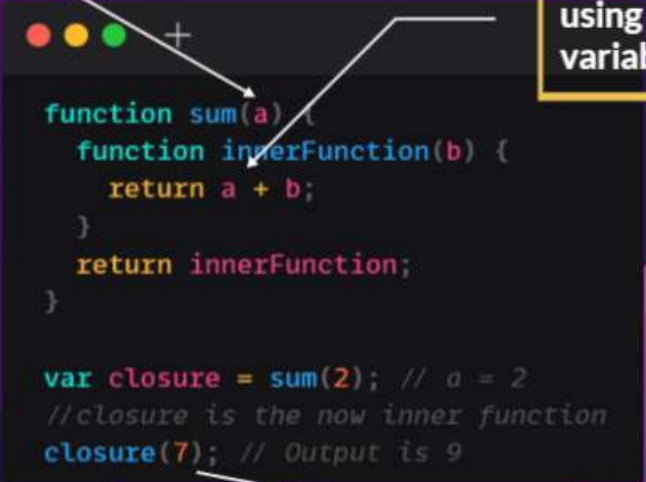
Code Example

```
function outerFunction() {  
  var outerVariable = 'I am from the outer scope';  
  
  function innerFunction() {  
    console.log(outerVariable);  
  }  
  
  return innerFunction;  
}  
  
var closure = outerFunction();  
closure(); // Outputs: "I am from the outer scope"
```

Accessing Outer Variables

Outer Variable "a"

inner
function
using
variable "a"



```
function sum(a) {  
  function innerFunction(b) {  
    return a + b;  
  }  
  return innerFunction;  
}  
  
var closure = sum(2); // a = 2  
//closure is the now inner function  
closure(7); // Output is 9
```

inner function "closure" has
access to variable "a"(2) even
after being returned from
outer function sum

Why Closure Occur?

- Closures form when an inner function retains a reference to variables in its outer function's scope.
- The closure, including the inner function and its referenced variables, is not removed by garbage collector and remains in memory even after the outer function completes execution.
- This enables the inner function to maintain functionality and access the necessary values from the outer scope accurately.

Uses Of Closure

- To Create Private Variables and Mehods

In Javascript, although we don't have a private keyword to create private variables and methods , we can utilize closure to create private variables and methods which are not directly accessible.

Example

```
var Amount = (function () {
  let accountBalance = 1000;
  let hasCreditCard = false; //private variable
  //private method
  function getCreditCard() {
    hasCreditCard = true;
  }

  return {
    getAmount: function () {
      console.log(accountBalance);
      console.log(hasCreditCard);
    },
    increaseAmount: function (amount) {
      accountBalance += amount;
      console.log(accountBalance);
    },
    decreaseAmount: function (amount) {
      accountBalance -= amount;
      console.log(accountBalance);
    },
  };
})();

Amount.increaseAmount(2000); //3000
Amount.decreaseAmount(500); //2500
Amount.getAmount(); //2500 false
Amount.getCreditCard(); //Error
```


Uses Of Closure

- Maintaining state between each function call

Closures help in maintaining the state between function calls without using a global variable. Closure come as a great alternative to the use of global variable in maintaining the state as global variables are prone to change anywhere in program.

Example

```
var greetVal = function greetFn() {  
  var greet = "How";  
  return function (val) {  
    greet = greet + val;  
    return greet;  
  };  
};  
  
var greeting = greetVal();  
  
console.log(greeting(" are")); //How are  
console.log(greeting(" you")); //How are you  
console.log(greeting(" John?")); //How are you John?
```

@psimran



Simran Bedi

Like Share and Follow